Встреча 5

Разработка микросервисных приложений на Golang

YADRO

Часто встречающиеся ошибки и замечания

- Зависимость “core” от адаптеров (db, rest)

Часто встречающиеся ошибки и замечания

- Зависимость “core” от адаптеров (db, rest)
- Получение статуса из gRPC ошибки (status.Code(err))

Часто встречающиеся ошибки и замечания

- Зависимость “core” от адаптеров (db, rest)
- Получение статуса из gRPC ошибки (status.Code(err))
- sql вместо sqlx (Select, Get)

Часто встречающиеся ошибки и замечания

- Зависимость “core” от адаптеров (db, rest)
- Получение статуса из gRPC ошибки (status.Code(err))
- sql вместо sqlx (Select, Get)
- Ненужные транзакции

Часто встречающиеся ошибки и замечания

- Зависимость “core” от адаптеров (db, rest)
- Получение статуса из gRPC ошибки (status.Code(err))
- sql вместо sqlx (Select, Get)
- Ненужные транзакции
- Отсутствие down миграции

Часто встречающиеся ошибки и замечания

- Зависимость “core” от адаптеров (db, rest)
- Получение статуса из gRPC ошибки (status.Code(err))
- sql вместо sqlx (Select, Get)
- Ненужные транзакции
- Отсутствие down миграции
- Select/Exec вместо SelectContext, ExecContext

Часто встречающиеся ошибки и замечания

- Зависимость “core” от адаптеров (db, rest)
- Получение статуса из gRPC ошибки (status.Code(err))
- sql вместо sqlx (Select, Get)
- Ненужные транзакции
- Отсутствие down миграции
- Select/Exec вместо SelectContext, ExecContext
- Много горутин :)

Часто встречающиеся ошибки и замечания

- Зависимость “core” от адаптеров (db, rest)
- Получение статуса из gRPC ошибки (status.Code(err))
- sql вместо sqlx (Select, Get)
- Ненужные транзакции
- Отсутствие down миграции
- Select/Exec вместо SelectContext, ExecContext
- Много горутин :)
- Concurrent update/drop/status

Часто встречающиеся ошибки и замечания

- Зависимость “core” от адаптеров (db, rest)
- Получение статуса из gRPC ошибки (status.Code(err))
- sql вместо sqlx (Select, Get)
- Ненужные транзакции
- Отсутствие down миграции
- Select/Exec вместо SelectContext, ExecContext
- Много горутин :)
- Concurrent update/drop/status
- select ctx.Done() в цикле - можно проще (и нужен ли?)

Context Done() vs Err()

```
for x range 5 {
    select {
        case <-ctx.Done():
            return ctx.Err()
        default:
    }
    // do one more iteration
}
```

```
for x range 5 {
    if ctx.Err() != nil {
            return ctx.Err()
    }
    // do one more iteration
}
```

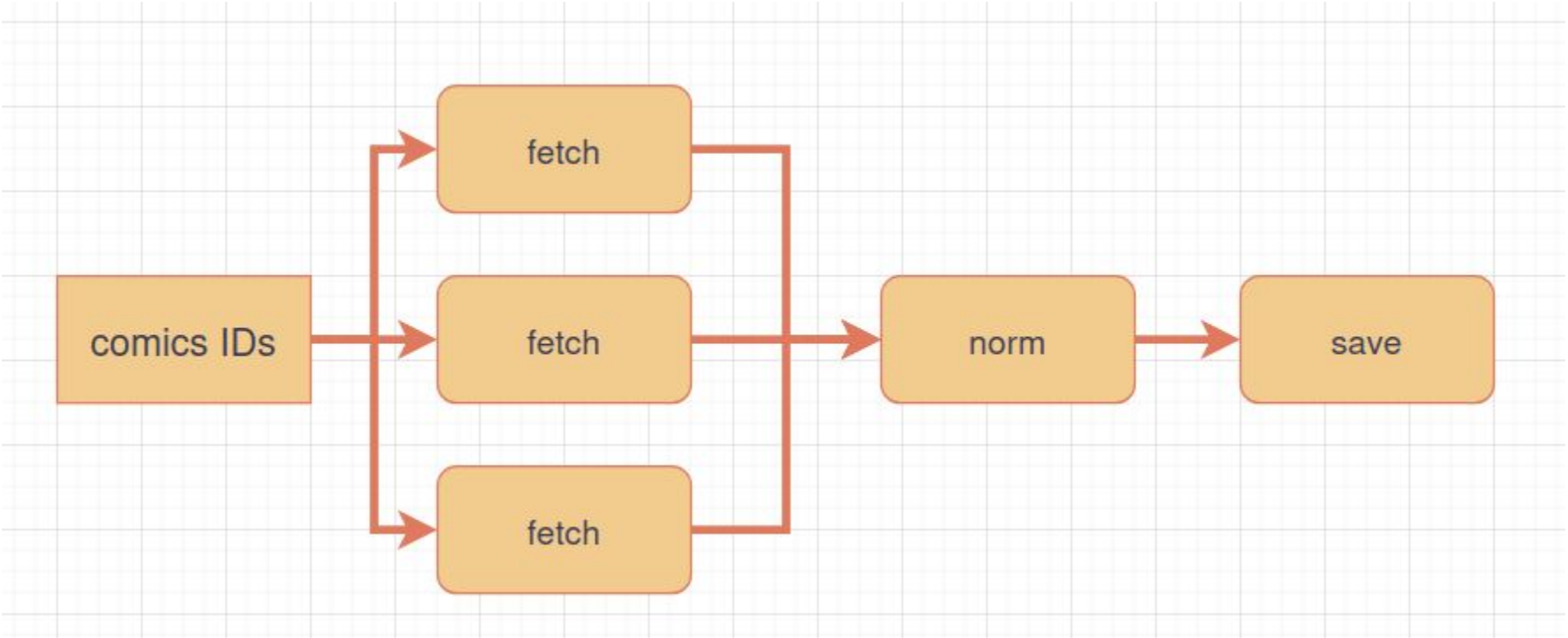
Часто встречающиеся ошибки и замечания

- Зависимость “core” от адаптеров (db, rest)
- Получение статуса из gRPC ошибки (status.Code(err))
- sql вместо sqlx (Select, Get)
- Ненужные транзакции
- Отсутствие down миграции
- Select/Exec вместо SelectContext, ExecContext
- Много горутин :)
- Concurrent update/drop/status
- select ctx.Done() в цикле - можно проще
- Lock вместо TryLock-а

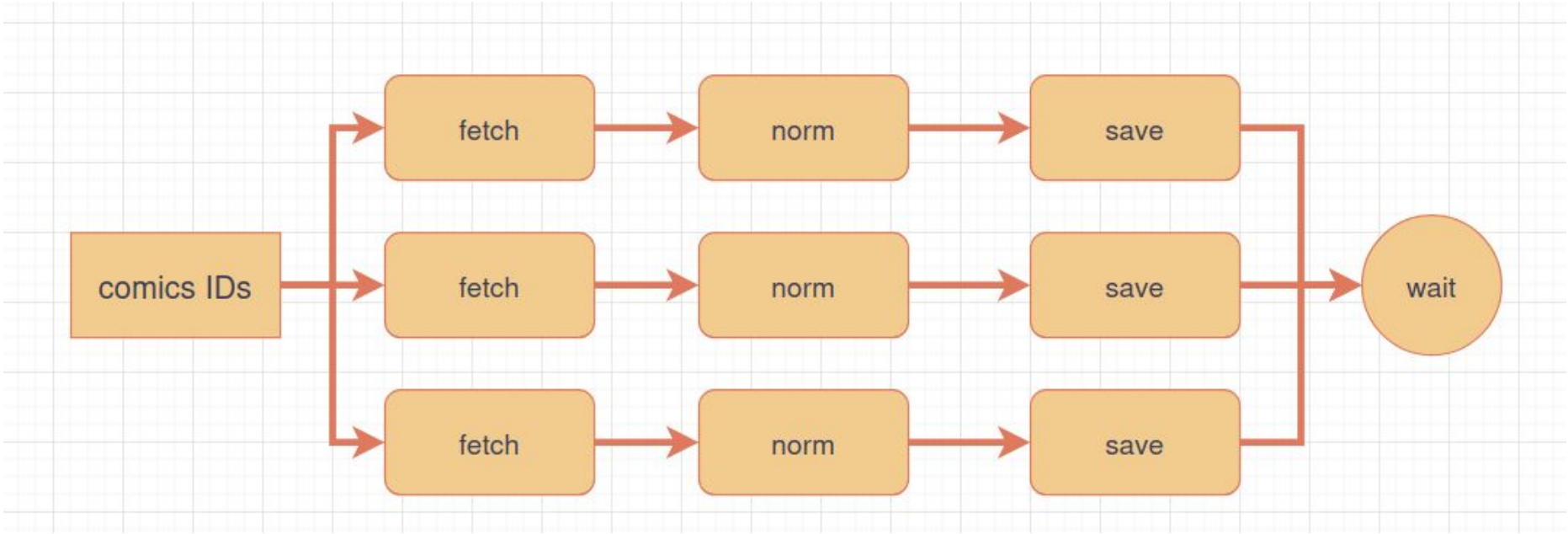
Часто встречающиеся ошибки и замечания

- Зависимость “core” от адаптеров (db, rest)
- Получение статуса из gRPC ошибки (status.Code(err))
- sql вместо sqlx (Select, Get)
- Ненужные транзакции
- Отсутствие down миграции
- Select/Exec вместо SelectContext, ExecContext
- Много горутин :)
- Concurrent update/drop/status
- select ctx.Done() в цикле - можно проще
- Lock вместо TryLock-а
- Статус можно атомиком, и даже без мьютексов вовсе

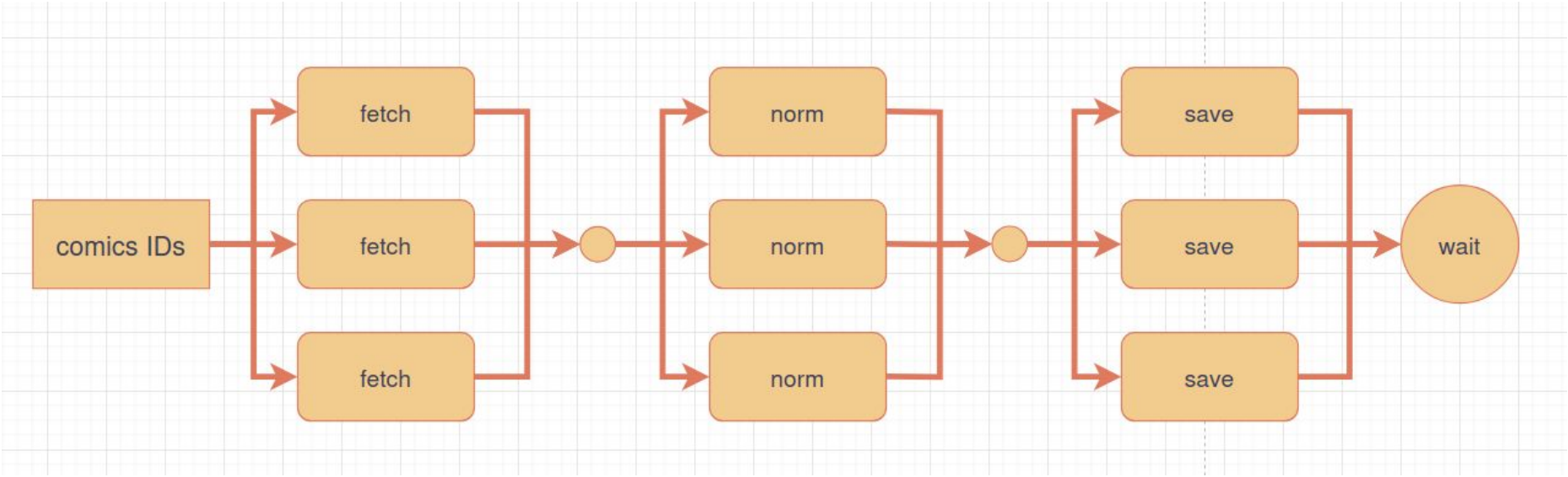
Многопоточка в Го: fan out->in



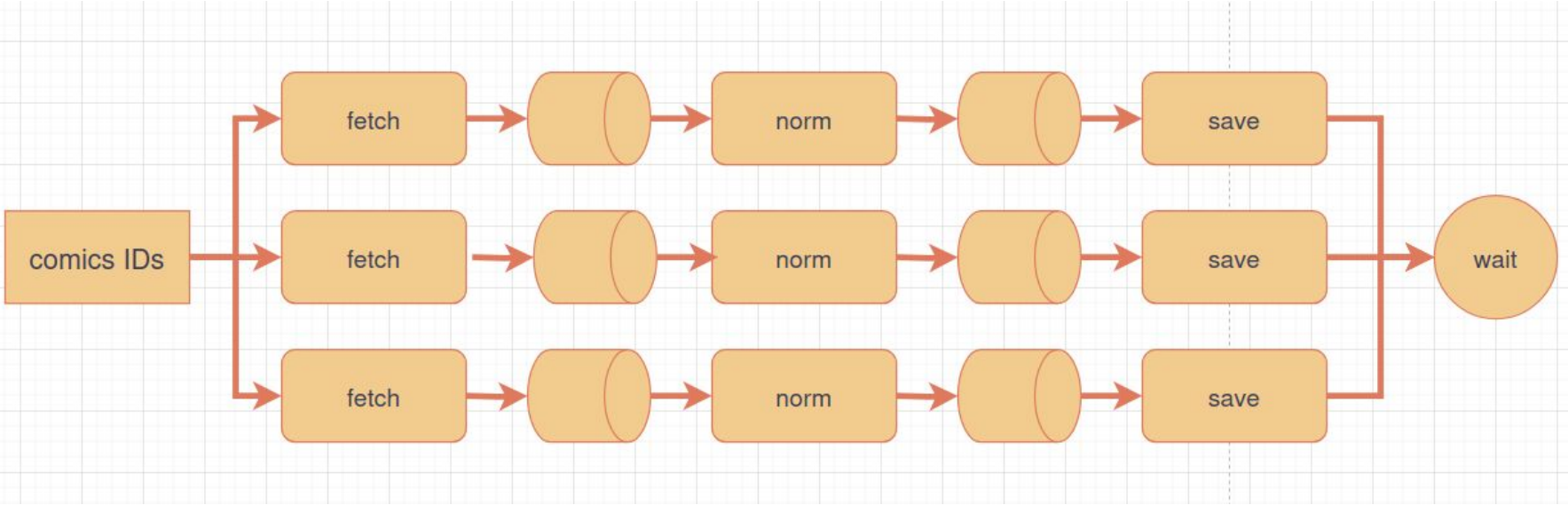
Многопоточка в Го: fan out->in + pipeline



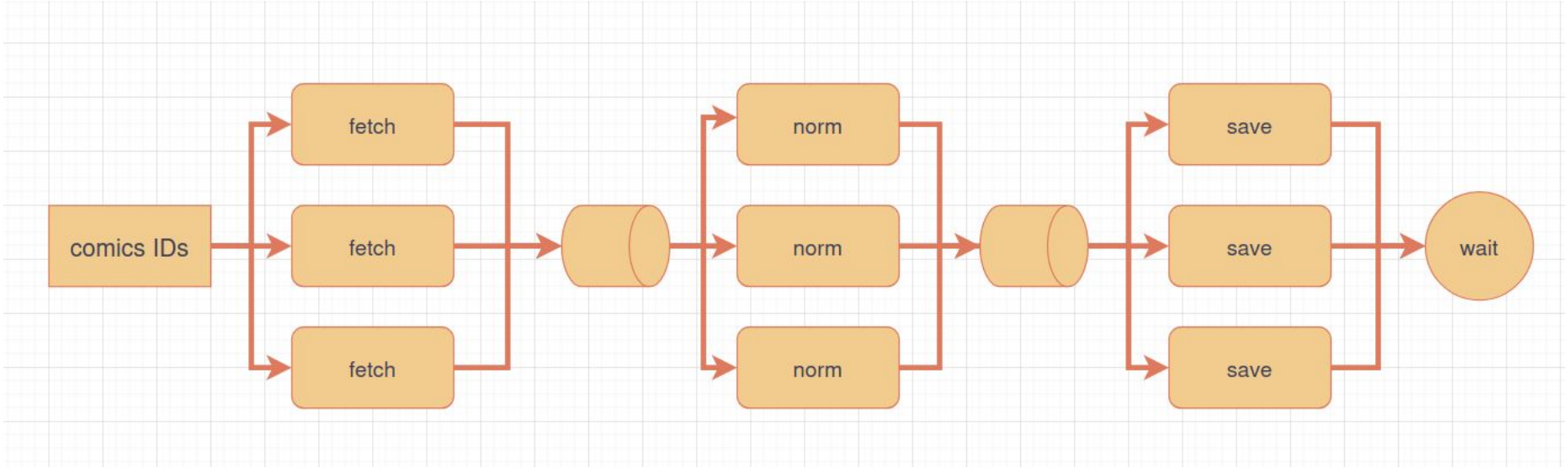
Многопоточка в Го: fan out -> in, fan out -> in, ..



Многопоточка в Го: fan out->in + pipeline + buffers



Многопоточка в Го: fan out->in->out->in … + buffers



Как организовать многопоточку по файлам / функциям

- чистые воркеры
  - не модифицируют вход
  - никаких wg, chan<- в аргументах
- заполняем, отчитываемся о выполнении и ждем wait group в одной функции
- По возможности вся логика открытия и закрытия каналов на одном экране